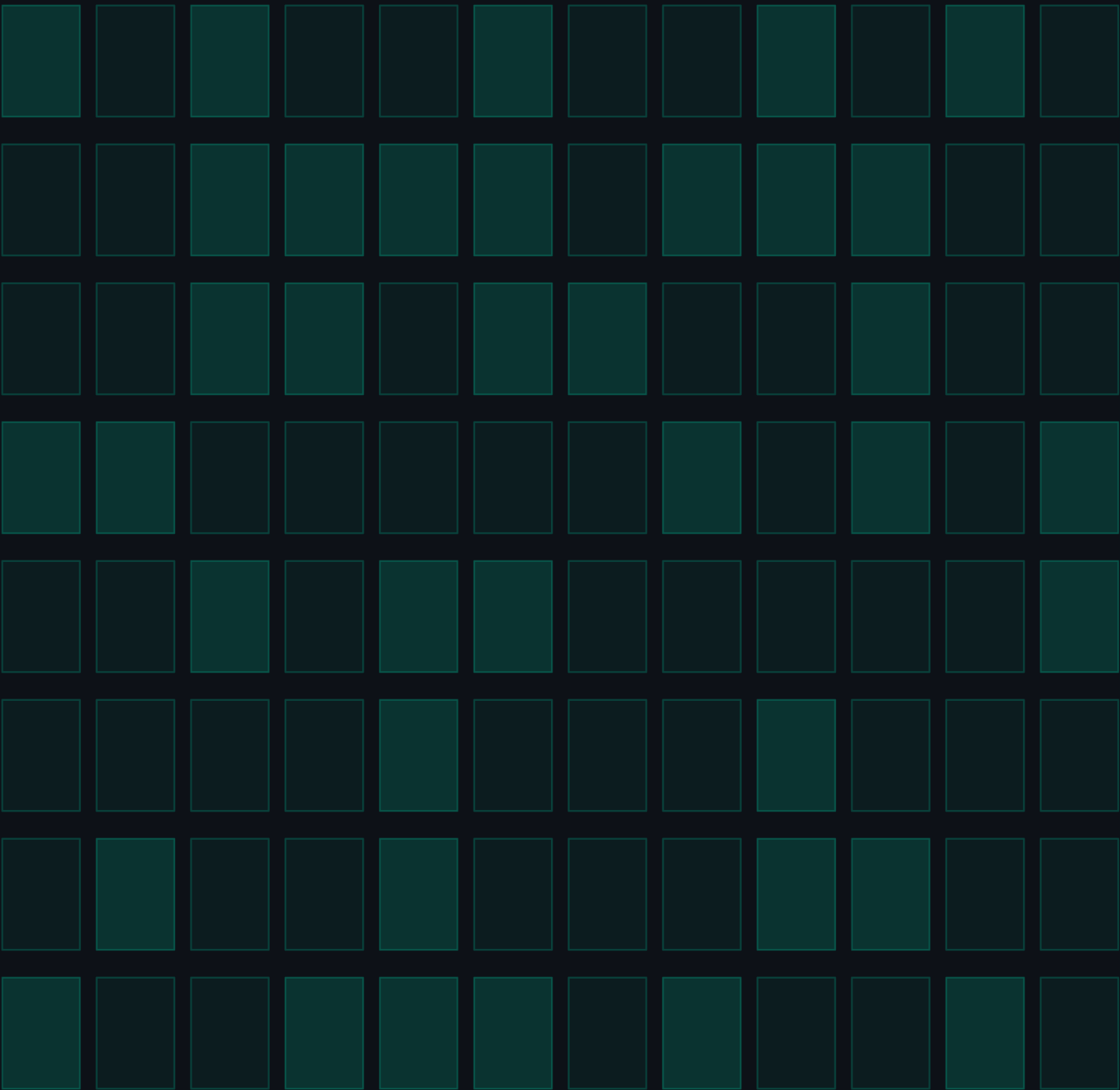


# StegoCipher

## Image Steganography System

*Hide messages in plain sight — pixel by pixel*

Sarah Mneja · Maryem Ammar · Ayoub El Atri  
IT360 – Information Assurance and Security | Year 2025–2026



# Agenda

01 Problem Statement

02 Core Concepts

03 Existing Solutions

04 Our Solution - StegoCipher

05 Use Case Diagram

06 LSB Algorithm

07 System Architecture & Components

08 Sequence Diagrams (Encode & Decode)

09 Tech Stack

10 Limitations

11 Enhancements

12 Live Demo and Conclusion

# 01 • Problem Statement

## The Core Challenge

Encryption protects the content of a message — but it reveals that something secret is being communicated.

*In surveillance-heavy environments, the mere presence of encrypted traffic can attract attention, put users at risk, or be blocked entirely.*

**What's needed is security through obscurity — hiding the very existence of the secret.**

- ▶ **Encryption alone: visible but unreadable**
- ▶ **Steganography alone: invisible but fragile**
- ▶ **Combined: hidden AND protected**

# 02 • Core Concepts



## Steganography

Concealing a secret message inside a non-secret carrier so the existence of the message stays hidden.  
From Greek: steganos (covered) + graphia (writing).

### Steganography vs Encryption

| Criteria       | Steganography            | Encryption             |
|----------------|--------------------------|------------------------|
| Purpose        | Hide existence of data   | Hide content of data   |
| Visibility     | Invisible carrier        | Visible but unreadable |
| Security alone | Low                      | Strong                 |
| Best practice  | Combined with encryption | Used alone or combined |

### Security Triangle

#### Imperceptibility

Invisible to human eye

#### Capacity

Max data that can be hidden

#### Robustness

Survives image transformations

*Improving one property typically degrades the others — a constant trade-off.*

# 03 • Existing Solutions

| Tool      | Interface     | Strength                        | Limitation                 | Encryption |
|-----------|---------------|---------------------------------|----------------------------|------------|
| OpenStego | GUI (desktop) | Easy to use, free & open-source | Limited features, dated UI | Basic      |
| Steghide  | CLI only      | Powerful embedding, compression | Not beginner-friendly      | Yes        |
| SilentEye | GUI (desktop) | User-friendly, plugin support   | Limited format support     | Partial    |

## Why Existing Tools Fall Short

- ▶ OpenStego — GUI but limited features
  - ▶ Steghide — powerful but CLI-only, no browser
  - ▶ SilentEye — beginner-friendly but inflexible
- **No modern, browser-based, key-secured option exists.**

# 04 • Our Solution - StegoCipher

*A client-side image steganography web app — zero servers, zero data leakage*



## No Server, No Leakage

100% client-side. Cover image, message, and key never leave the device.  
Strongest possible privacy guarantee.



## Key-Based Pixel Shuffling

Polynomial rolling hash → LCG seed → Fisher-Yates shuffle.  
Without the correct key, the bit sequence cannot be reconstructed.



## Lossless PNG Output

Stego image always exported as PNG.  
JPEG re-encoding destroys LSBs — JPEG output is never offered.



## Alpha Channel Untouched

Only R, G, B channels used. Alpha (every 4th byte) is always preserved,  
preventing trivial alpha-plane analysis.



## 32-bit Length Header

Message length is hidden inside the bitstream. An attacker cannot learn the  
payload size without successful extraction.



## Wrong-Key Detection

TextDecoder called with { fatal: true }.  
Invalid UTF-8 from wrong key throws an exception  
→ UI shows error, not garbage.

# 04 • StegoCipher

## Why StegoCipher stands out



Runs entirely in the browser — no install, no server, no data leakage



Key-based LSB shuffling defeats basic statistical steganalysis

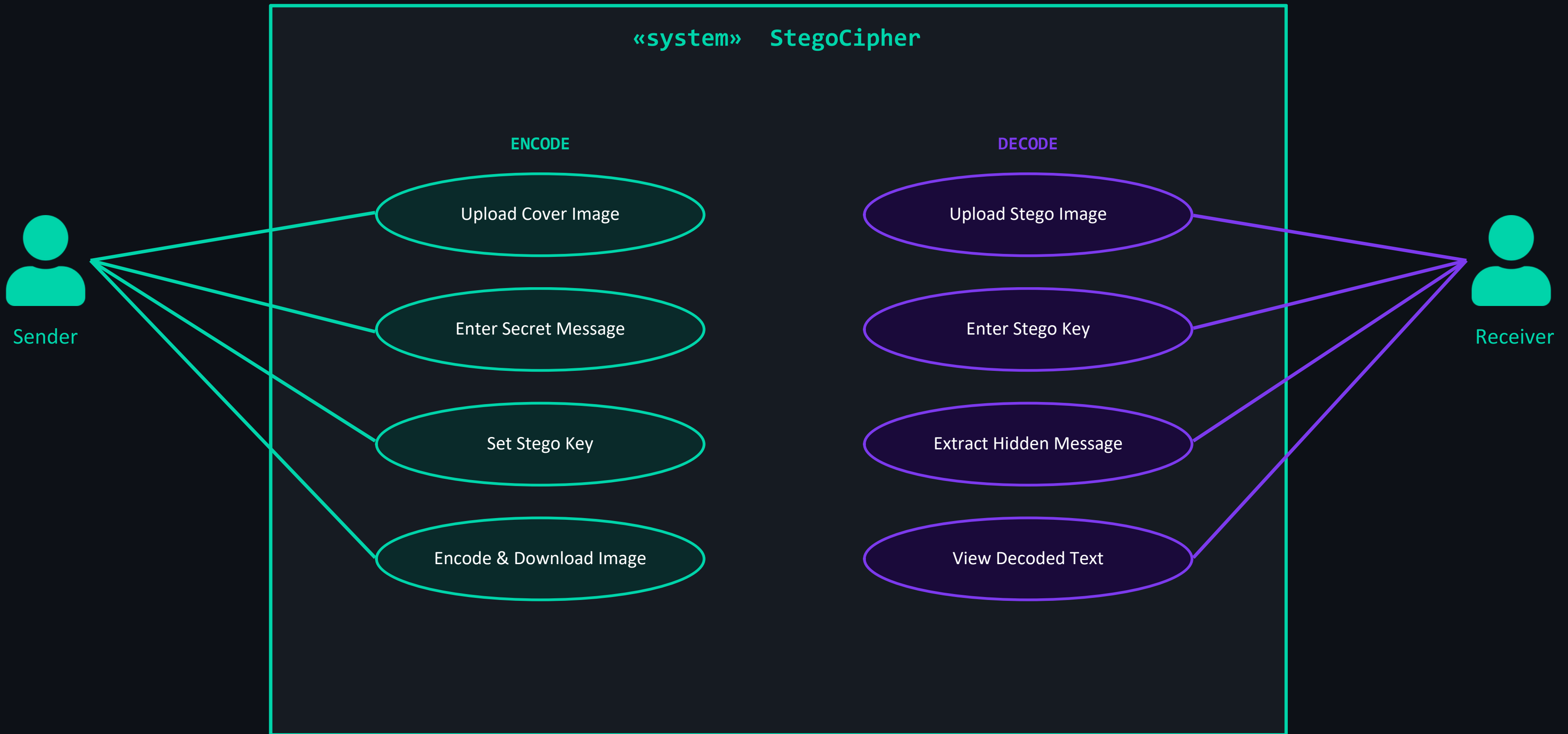


Live capacity bar, drag-and-drop UX, wrong-key detection with clear error messages



Single self-contained HTML file — share it like a document

# 05 • Use Case Diagram



# 06 • LSB Algorithm

1

## Cover Image

Original PNG/BMP loaded into browser canvas as RGBA pixel array

2

## Message → Binary

UTF-8 text encoded via TextEncoder → flat bit array (MSB first)

3

## Key → Pixel Order

Password hashed → LCG seed → Fisher-Yates shuffle of channel indices

4

## LSB Substitution

Each message bit replaces the LSB of one R, G or B channel value

5

## Stego PNG Output

Modified pixel array exported as lossless PNG — visually identical

# 06 • LSB Algorithm - Deep Dive

## How One Bit Changes a Pixel

Original pixel: 10110010

Modified pixel: 1011001 **1**

↑ Only the last bit (LSB) changes → visually undetectable

## Capacity Formula

$\text{Capacity} = W \times H \times 3 \text{ bits}$

512×512 RGB image → 786,432 bits ≈ 96 KB of hidden data

## RGB Channel Embedding

**R**

xxxxxxx

**1**

← msg bit

3 bits/pixel

**G**

xxxxxxx

**0**

← msg bit

3 bits/pixel

**B**

xxxxxxx

**1**

← msg bit

3 bits/pixel

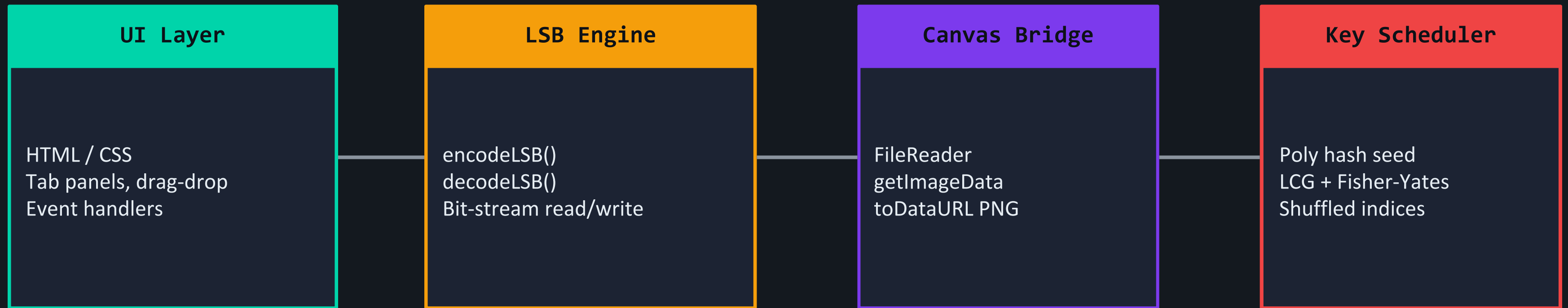
## Role of the Stego Key

Without key: bits inserted sequentially → trivially detectable by chi-square analysis.

With key: PRNG shuffles pixel order → bits scattered uniformly → defeats statistical detection.

# 07 • System Architecture & Components

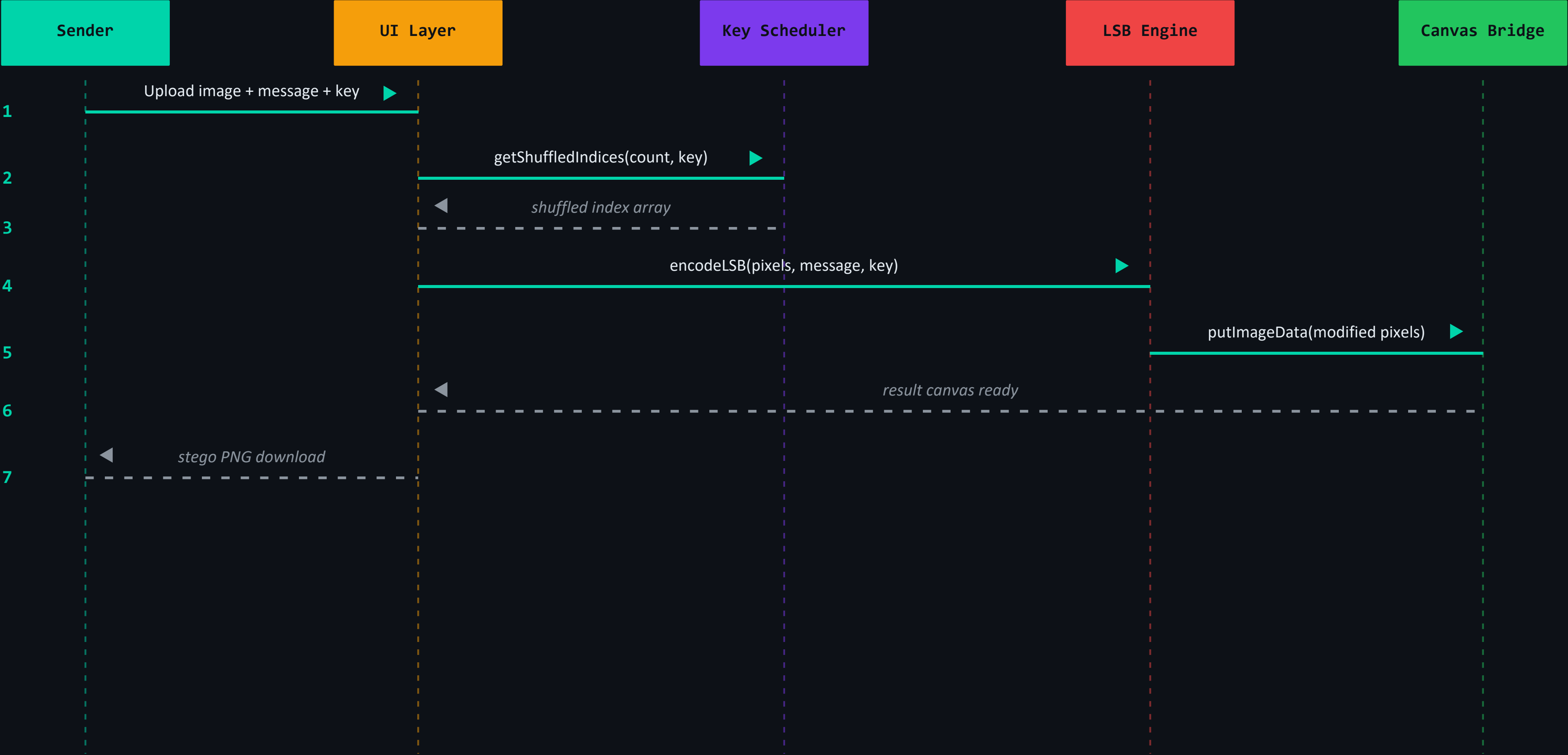
USER BROWSER (all processing is local – no server)



Storage Layer: None (stateless) — no localStorage, no cookies, no server calls. Cover image & message never leave the device.

*Single-page, client-side-only SPA • Pure Vanilla JS • Single self-contained HTML file • No build step • No npm*

# 08 • Sequence Diagram – Encoding Phase



# 08 • Sequence Diagram – Decoding Phase



Wrong-key detection: `TextDecoder({ fatal: true })` throws on invalid UTF-8 → UI shows an error, never garbage text.

# 09 • Tech Stack

## Technology Stack

### HTML5 Canvas API

Pixel-level read/write via `getImageData` / `putImageData`

### TextEncoder/Decoder

UTF-8 ↔ bytes; fatal mode for wrong-key detection

### Vanilla JS (ES2020)

All logic: UI, LSB engine, key scheduler — no frameworks

### CSS3 + Google Fonts

Dark theme, animations, Syne + Space Mono typography

### HTML5 File API

Upload images via file picker and drag-and-drop

### Netlify (optional)

Static hosting — also works opened directly from disk

# 10 • Limitations

## !! Not Robust to Compression

LSB data is destroyed by any lossy compression (JPEG, social media re-encoding). The stego PNG must reach the recipient unmodified.

## !! Statistical Detectability

Without a key (sequential mode), chi-square steganalysis on the LSB plane can detect embedded data. Key-mode mitigates this but does not eliminate it.

## !! Capacity vs Image Size

Capacity is strictly bounded:  $W \times H \times 3$  bits. Small or low-resolution images can only hide short messages.

## !! No Encryption Layer

The hidden message is hidden but not encrypted. A successful extraction gives plaintext. Users must encrypt content before embedding for full security.

## !! Browser Memory Limit

Very large images (e.g. 50 MP RAW) may exhaust browser canvas memory on low-end devices.

## !! Key Distribution Problem

The stego key must be shared securely out-of-band. If the key channel is compromised, the system provides only obscurity.

# 11 • Enhancements

[AES]

## AES-256 Encryption Layer

Encrypt the secret message with AES-GCM before embedding — adding real cryptographic protection on top of steganographic hiding.

[FILE]

## File Payload Support

Extend beyond text to hide binary files (PDFs, ZIPs, images) inside the cover image, with MIME type stored in the header.

[STEG]

## Steganalysis Resistance

Implement adaptive LSB techniques (e.g., LSBMR, PVD) to defeat chi-square and RS analysis that can detect sequential embedding.

[MEDIA]

## Multi-Image & Audio Carriers

Support audio files (WAV) and video frames as alternative carriers, broadening the range of covert channels.

[PWA]

## Progressive Web App (PWA)

Package as an installable PWA with offline support so users can run StegoCipher without any network access.

[METRICS]

## PSNR / SSIM Quality Metrics UI

Display real-time image quality scores (Peak Signal-to-Noise Ratio, SSIM) after encoding to quantify imperceptibility.

# 12 • Live Demo & Conclusion

## Demo Scenarios

### Scenario 1

Short text in a small image

### Scenario 2

Long paragraph in a hi-res photo

### Scenario 3

Wrong key → clear error message

Live app: [StegoCipher](#)

## Key Takeaways

- ✓ LSB steganography is imperceptible but fragile — lossless formats (PNG) are mandatory
- ✓ Key-based pixel shuffling significantly raises the bar against statistical steganalysis
- ✓ Client-side architecture maximises privacy: zero data ever reaches a server

Thank You!



StegoCipher - Image Steganography  
System

Sarah Mneja · Maryem Ammar · Ayoub El Atri  
IT360 – Information Assurance and Security | Year 2025–2026

